Distributed Movie Booking System — Architecture Blueprint

Project codename: SeatLock **Goal:** Prove — with working code and benchmark data — that this system guarantees zero double-booking under high concurrency, stays consistent across distributed services, degrades gracefully under partial failure, and scales measurably.

Design philosophy: 5 services, no more. Every component exists to answer one of the 10 hard questions. If a technology doesn't map to a specific failure mode you're defending against, it doesn't go in the stack.

---

1. Service Boundaries

Service	Responsibility	Owns
Booking Service	Public API, booking lifecycle, orchestrates the saga	Booking records, Saga state
Inventory Service	Seat state, concurrency control, seat locking	Seat/show inventory, version numbers
Payment Service	Simulated payment gateway with configurable failure/latency injection	Payment transactions
Notification Service	Confirmation emails/SMS (simulated)	Notification log
Saga Orchestrator	Drives the booking workflow state machine, handles compensation	Saga instance state

Why orchestration over choreography: with 5 services, a choreographed saga means 5 services each partially know the workflow — very hard to draw and defend in an interview whiteboard session. One orchestrator owning the state machine gives you a single place to point to and say "here is exactly what happens on every failure," which is the entire point of this project.

Communication:

- Sync: REST/gRPC for client-facing Booking API, and Orchestrator → Inventory/Payment for the "do this now" calls.
- Async: Kafka for events that other services react to (booking confirmed, booking failed, payment completed) — used for the Outbox relay and Notification triggering, not for the core saga control flow (control flow needs low latency + easy tracing; events are for side effects).

---

2. The Core Problem: Zero Double-Booking (Q1 + Q2)

Three options, and why I'm picking one

Approach	How it works	Pros	Cons	Verdict
A. DB row lock (`SELECT ... FOR UPDATE`)	Postgres row lock on the seat row for the transaction duration	Simple, strongly consistent, no extra infra	Doesn't scale across horizontally-partitioned DBs; long lock hold times under load kill throughput	Use as the **final commit guard**, not the primary mechanism
B. Redlock (Redis distributed lock)	Acquire a lock key per seat across N Redis nodes before touching DB	Fast, works across pods	Redlock has known correctness edge cases (clock drift, GC pauses) — Martin Kleppmann's critique is real and you should know it exists	Use as a **short-lived intent lock**, not the source of truth
C. Optimistic concurrency (version column)	Read seat + version, attempt UPDATE with `WHERE version = ?`, retry on conflict	No lock contention, scales horizontally, DB is still the source of truth	Requires retry logic; wasted work on conflict under extreme contention	**Primary mechanism**

Chosen design: Optimistic concurrency as the source of truth, Redis lock as a fast-fail pre-check.

Flow for reserving a seat:

1. Client requests seat hold → Inventory Service.
2. Inventory Service attempts a Redis `SETNX seat:{showId}:{seatId} = requestId EX 120` (2-min TTL). If this fails, seat is already being held by someone else → **fast fail in <5ms without touching the DB at all**. This is what lets you survive 10,000 concurrent requests on the same popular seat without hammering Postgres.
3. If Redis lock acquired: read seat row (`status`, `version`) from Postgres.
4. `UPDATE seats SET status='HELD', version=version+1 WHERE seat_id=? AND version=? AND status='AVAILABLE'`
5. If 0 rows affected → someone else committed between steps 3-4 (race window) → release Redis lock, return 409 Conflict.
6. If 1 row affected → hold succeeds, TTL-based hold, proceed to payment.

This answers Q1 directly: **Redis lock gives you the throughput (rejects 9,999 of 10,000 concurrent requests in milliseconds), the DB version check gives you the correctness**

guarantee (even if two requests somehow both pass the Redis check due to a Redis failover edge case, only one wins the `UPDATE`).

This answers Q2: the lock isn't pod-local — it's in Redis, so it doesn't matter which of your N Kubernetes pods handles the request. Statelessness of the pods is preserved.

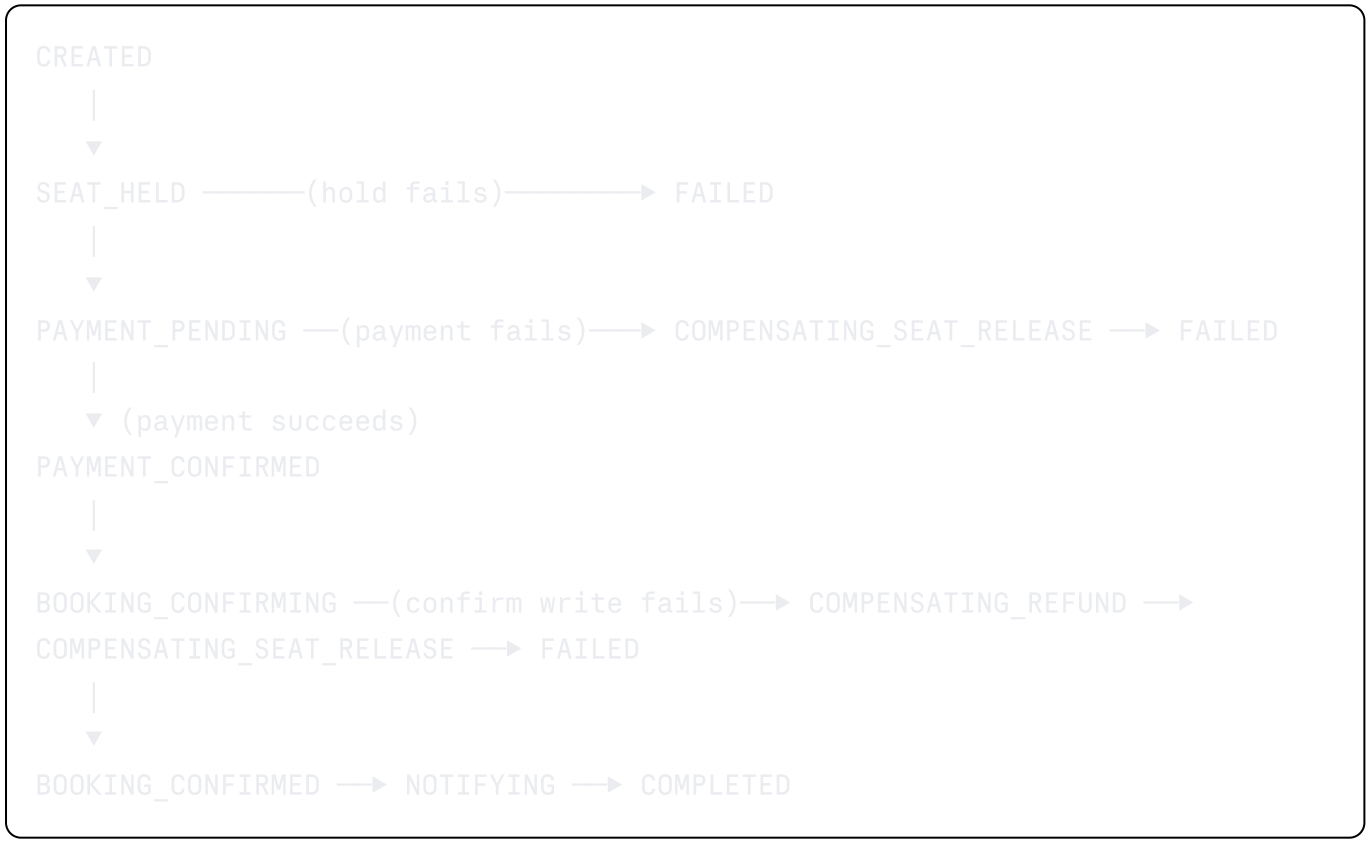
Proof artifact for this section: a k6 script that fires 500 concurrent requests at the *same seat* and asserts `successful_holds == 1`. This is your headline benchmark — screenshot it, put it in the README.

---

3. What Happens When Payment Succeeds but Booking Confirmation Fails (Q3)

This is the classic distributed transaction problem. You are explicitly **not** using 2PC (two-phase commit) — say this out loud in interviews, and say why: 2PC blocks on coordinator failure and doesn't play well with Kafka/async systems. You're using **Saga with compensation**.

Booking Saga — State Machine



Key design point: **compensation is ordered in reverse** — refund before releasing the seat, because if you release the seat first and refund fails, you could resell a seat someone already paid for. This ordering detail is exactly the kind of thing that shows engineering maturity in an interview.

Concretely, for Q3: payment succeeds but the booking confirmation DB write fails (e.g., pod crash mid-transaction). Because the Orchestrator persists saga state *before* each step (not after), on restart it recovers the saga at `PAYMENT_CONFIRMED` and knows it must retry `BOOKING_CONFIRMING` or trigger compensation if the retry budget is exhausted. This is why

saga state must itself be durable and reloadable — it's the source of truth for "what do I do next" after a crash.

---

4. Reliable Event Publishing After a DB Transaction — Outbox Pattern (Q4)

The problem: if you `COMMIT` a booking to Postgres and then separately publish a Kafka event, a crash between those two steps loses the event forever, or publishes an event for a transaction that never committed.

Solution: Transactional Outbox.

1. In the *same DB transaction* as the booking state change, insert a row into an `outbox_events` table (`id, aggregate_id, event_type, payload, created_at, published_at NULL`).
2. A separate **Outbox Relay** process (can be Debezium CDC reading the Postgres WAL, or a simple polling publisher for a leaner build) reads unpublished rows, publishes to Kafka, then marks `published_at`.
3. If the relay crashes after publishing but before marking `published_at`, it republishes on restart → **this is why every consumer must be idempotent** (ties directly into Q5).

For your resume, I'd build the polling version first (simpler to explain and demo), then explicitly note in your README "production version would use Debezium CDC to avoid polling latency and DB load" — showing you know the trade-off even if you didn't build the fancier version. That's a legitimate and common thing senior engineers do: pick the simpler correct thing, document the alternative.

---

5. Handling Duplicate Kafka Events (Q5)

At-least-once delivery is unavoidable with the outbox pattern (relay crash → republish) and with Kafka consumer rebalances. So: **every consumer must be idempotent by design.**

Two idempotency layers:

1. **Event-level:** each outbox event carries a unique `event_id` (UUID). Consumers keep a `processed_events (event_id PK, processed_at)` table and check-then-insert before acting. `INSERT ... ON CONFLICT DO NOTHING` gives you an atomic idempotency check.
2. **Business-level:** the Notification Service, for example, doesn't just dedupe by event_id — it also checks `booking_id` hasn't already been notified, as defense in depth in case event_id generation ever has a bug.

Proof artifact: a chaos test that force-republishes the same Kafka event 3x and asserts the downstream side effect (e.g., notification count, seat status) only happened once.

---

6. Dependency Failure Matrix (Q6)

This table *is* your system design interview answer sheet. Build it as a literal markdown table in your repo and make sure every row is backed by a test.

Dependency down	Detection	System behavior	User-facing result
Redis down	Connection failures on lock attempt	Fall back to DB-only optimistic locking (degraded but correct — slower, no fast-fail, but still zero double-booking) with a circuit breaker to avoid retry storms against dead Redis	Slightly slower booking, still succeeds
Kafka down	Outbox relay publish failures	Events queue up in `outbox_events` (unpublished), booking flow itself is unaffected since it doesn't wait on Kafka synchronously	Booking succeeds; notifications delayed until Kafka recovers
Payment service down/timeout	Circuit breaker opens after N consecutive failures	Saga stays in `PAYMENT_PENDING`, retries with backoff; after max retries → compensate (release seat)	Booking fails cleanly, seat released, user notified to retry
Postgres (Inventory DB) down	Connection pool exhaustion / health check fail	Reject new holds immediately (fail-fast, not hang) via circuit breaker; existing Redis-held seats keep their TTL so they don't leak forever	503 with retry-after, no data corruption
Orchestrator pod crashes mid-saga	K8s liveness probe restarts pod	New pod instance reloads persisted saga state from DB and resumes at last durable step	Slight delay, no lost or duplicated bookings

This single table, backed by actual `docker kill` / chaos tests with logs, is worth more in an interview than almost anything else in the project.

---

7. Preventing a Retry Storm (Q7)

Naive retries under failure make things worse (every client retries → load spikes exactly when the dependency is already struggling). Three layered defenses:

1. **Exponential backoff with jitter** on every retry (Orchestrator → Payment, Relay → Kafka). Never fixed-interval retry.
2. **Circuit breaker** (e.g., Resilience4j) around Payment and Redis calls: after N failures in a window, open the circuit and fail fast for a cooldown period instead of continuing to hammer a struggling dependency.
3. **Bulkhead / rate limiting** at the API gateway layer — cap concurrent in-flight requests per client and per downstream dependency so one misbehaving caller or one degraded dependency can't exhaust the whole thread/connection pool.

Proof artifact: a graph from your load test showing request latency/error rate *with* circuit breaker vs *without* — this visual is extremely compelling in a portfolio README.

---

8. Blockbuster Traffic Spike (Q8)

This is a capacity/backpressure problem, not a correctness problem — don't try to solve it with more locking.

- **Virtual waiting room / queue-based load leveling:** when demand for a specific show exceeds a threshold, incoming requests are queued (a Kafka topic or Redis-backed queue works fine) and released to the booking flow at a rate the system can sustain, rather than all hitting Inventory Service simultaneously.
- **Horizontal autoscaling** on Inventory and Payment pods via HPA, keyed on CPU + custom Kafka consumer lag metric.
- **Read/write separation:** seat availability *display* (read path) is served from a cache with short TTL, not from the strongly-consistent write path — showing seats as "likely available" is fine for browsing; the write path (actual hold) is where correctness matters.

---

9. Observability Across 5 Services (Q9)

- **Correlation ID / trace ID** generated at the API gateway, propagated through every service call and into every Kafka event header. Without this, "debug a failed booking across 5 services" is impossible — this is the single highest-leverage thing to build first.
- **Distributed tracing** via OpenTelemetry + Jaeger (or Zipkin) — one trace per booking request, spanning Booking API → Orchestrator → Inventory → Payment → Notification.
- **Structured logging** (JSON logs, not printf) with `trace_id`, `booking_id`, `saga_state` on every log line, shipped to a simple ELK/Loki stack.
- **Metrics** (Micrometer + Prometheus + Grafana): booking success rate, saga state distribution (how many bookings currently sit in each state — great for spotting stuck sagas), p50/p95/p99 latency per service, circuit breaker open/closed state.

Proof artifact: a Grafana dashboard screenshot, and a walkthrough in your README: "here is a trace_id for a booking that failed at the payment step — here's the Jaeger trace, here's the log line, here's the compensating transaction that fired."

---

10. Load Testing Plan (Q10)

Use **k6** (scriptable, good reporting, CI-friendly) or Gatling.

Scenario	What it proves
10,000 concurrent requests for the *same seat*	Zero double-booking (exactly 1 success)
Sustained 500 req/s for 10 min across many seats/shows	Baseline throughput, p95/p99 latency under normal load
Spike test: 50 → 5,000 req/s in 10s	Autoscaling response time, queueing behavior under blockbuster-release conditions
Soak test: 200 req/s for 2 hours	Memory leaks, connection pool exhaustion, TTL cleanup correctness
Chaos: kill Payment service mid-test	Saga compensation correctness, no stuck sagas, no lost seat holds
Chaos: kill Redis mid-test	Fallback-to-DB-locking correctness, no double-booking during the transition

Every scenario should produce a report (k6 has built-in JSON/HTML output) that goes into `/docs/benchmarks/` in your repo, dated and versioned. This is the artifact that makes question 10 a "yes, and here's the file" instead of a claim.

---

11. Data Model (Core Tables)

```sql
```

```
-- Inventory Service
seats (
  seat_id UUID PK,
  show_id UUID,
  seat_number TEXT,
  status TEXT CHECK (status IN ('AVAILABLE','HELD','BOOKED')),
  version INT NOT NULL DEFAULT 0,
  held_until TIMESTAMPTZ
)

-- Booking Service
bookings (
  booking_id UUID PK,
  user_id UUID,
  show_id UUID,
  seat_ids UUID[],
  status TEXT, -- mirrors saga terminal states
  created_at TIMESTAMPTZ
)

-- Saga Orchestrator
saga_state (
  saga_id UUID PK,
  booking_id UUID,
  current_step TEXT,
  payload JSONB,        -- context needed to resume/compensate
  retry_count INT,
  updated_at TIMESTAMPTZ
)

-- Outbox (lives in each service that publishes events)
outbox_events (
  event_id UUID PK,
  aggregate_id UUID,
  event_type TEXT,
  payload JSONB,
  created_at TIMESTAMPTZ,
  published_at TIMESTAMPTZ NULL
)

-- Consumer-side idempotency
processed_events (
  event_id UUID PK,
  processed_at TIMESTAMPTZ
)
```

12. Kafka Event Contracts

Topic	Producer	Consumer(s)	Key event types
`booking.events`	Booking Outbox Relay	Notification Service, Analytics (optional, skip if not building)	`BookingConfirmed`, `BookingFailed`
`payment.events`	Payment Outbox Relay	Saga Orchestrator	`PaymentSucceeded`, `PaymentFailed`
`inventory.events`	Inventory Outbox Relay	Saga Orchestrator	`SeatHeld`, `SeatReleased`

Event envelope (every event, every topic):

```json
{
  "event_id": "uuid",
  "event_type": "PaymentSucceeded",
  "trace_id": "uuid",
  "aggregate_id": "booking-uuid",
  "occurred_at": "ISO8601",
  "payload": { }
}
```

Partition key = `aggregate_id` (booking_id) — guarantees ordering of events for the same booking, which the Orchestrator depends on.

---

13. Tech Stack (deliberately restrained)

- **Language:** Java 21 + Spring Boot 3.3 (matches your existing stack — reinforces depth)
- **DB:** PostgreSQL (row locks + version column)
- **Cache/Lock:** Redis
- **Messaging:** Kafka
- **Resilience:** Resilience4j (circuit breaker, retry, bulkhead)
- **Observability:** OpenTelemetry + Jaeger, Prometheus + Grafana, structured JSON logs
- **Load testing:** k6
- **Orchestration:** Kubernetes (Minikube/Kind is fine for demo) — this is what makes Q2 ("across multiple pods") a real test, not a hypothetical
- **CI:** GitHub Actions running the k6 zero-double-booking test on every PR — this alone is a strong signal, most portfolio projects have zero automated correctness verification

Explicitly excluded, and worth stating why in your README: Elasticsearch (no search problem here), AI/recommendations (out of scope, would dilute the story), 20 microservices (5 is enough to demonstrate every pattern; more would just add noise).

---

14. What You Need to Learn (in build order)

1. Postgres `SELECT FOR UPDATE` vs optimistic locking — trade-offs, `WHERE version=?` pattern
2. Redis `SETNX`/`SET NX EX` semantics, TTL-based locks, and *why* Redlock is controversial (read Kleppmann's critique — you'll want to reference it)
3. Saga pattern: orchestration vs choreography, compensation ordering
4. Transactional Outbox pattern + (optionally) Debezium CDC basics
5. Kafka consumer idempotency, partition keys, consumer group rebalancing behavior
6. Resilience4j: circuit breaker states (closed/open/half-open), retry + backoff + jitter, bulkhead
7. OpenTelemetry basics: spans, trace propagation across HTTP + Kafka headers
8. k6 scripting: scenarios, thresholds, HTML reporting
9. Kubernetes basics you don't already have: HPA, liveness/readiness probes, ConfigMaps for per-env config
10. Chaos testing approach: even simple `docker kill` / `kubectl delete pod` scripted into your test suite counts — you don't need Chaos Mesh/Litmus for this to be credible, but mention them as the production-grade option

---

15. Build Order (suggested milestones)

1. Inventory Service + optimistic locking + Redis pre-check, with the 500-concurrent-same-seat k6 test passing → **this alone is already a strong demo**
2. Add Saga Orchestrator + Payment Service (mocked) + compensation flow
3. Add Outbox + Kafka + idempotent consumers
4. Add Resilience4j circuit breakers + retry/backoff, write the failure matrix tests
5. Add observability (tracing/metrics/logs) — retrofit trace_id propagation everywhere
6. Full load test suite + chaos tests, generate the benchmark reports
7. Write the architecture decision records (ADRs) — one per major decision (locking strategy, saga vs 2PC, outbox vs dual-write) — this is what turns "I built X" into "I can defend X"

Closing note

The single highest-leverage thing you can do differently from every other candidate with a "booking system" on their resume: **ship the `/docs/benchmarks/` folder with real numbers and the `/docs/adr/` folder with real trade-off writeups.** Almost nobody does this. The code proves you can build it; the docs prove you understand *why* it works.